The Road Repair Problem

Jose Luna

December 2024

1 Introduction

Consider the following *Road Repair Problem*: we are given a network of roads (with capacities), as well as one source node, one destination node, and several transshipment nodes in between. The capacity of the network is defined as the maximum in-flow that can be achieved at the destination node, if we let the source node have an infinite supply. Each roads needs to be repaired, and the length of each repair job depends on the specific road. Multiple roads can be repaired at once, but repairing them closes the road, which will affect the capacity of the network. Our aim is to define a repair schedule such that the total repair time is minimized, while ensuring that the total capacity of the network remains above a certain threshold at all times, from $t = 1$ to $t = T$, where T is a predefined time horizon.

We will decompose the problem into a *Preprocessing Step*, then a *Master Problem* and a *Subproblem*.

- *Preprocessing Step*: First, we identify a set of *critical road subsets* (subsets of the roads whose removal bring the maximum flow below the threshold). These do not need to be exhaustive (in fact, an exhaustive search is intractible for large problems, since the number of subsets grows exponentially). The purpose is to give the Master Problem a set of constraints (disallowed repair combinations) as a sort of "warm start". These critical road subsets are found by enumerating the length-1 and length-2 subsets of the roads, and running the Edmonds-Karp algorithm to determine whether the maximum flow of the graph with these roads removed is above the threshold.

- *Master Problem*: The Master Problem consists of finding a repair schedule that minimizes total repair time, while disallowing the previously found critical subsets (for each subset, we will enforce no overlap for these repair jobs). This is done via Constraint Programming

- *Subproblem*: For each schedule produced by the Master Problem, there will be at most T topologically different networks whose flow we must check. For each of these networks, the Subproblem consists of checking whether the capacity constraint is satisfied. If the capacity constraint is not satisfied for any one of the networks, we generate a new set of constraints to be added to the Master problem. The new constraint disallows each specific critical subset from overlapping. It is sometimes useful to disallow large subsets of each critical subset as well. Though this constrains the scheduling problem more, in practice I found it to lead to faster solutions.

2 Master Problem

Index Use

$r \in R$: index on roads

$c \in C \subseteq \mathcal{P}(R)$: index on critical road subsets. Each critical subset is a subset of R, and so C is a subset of the *power set* of R.

$t \in \{1, ..., T\}$: index on timesteps

Data

t_r: duration of repair job r (integer)

T: time horizon (integer): this can be set by adding up all durations, as this is the maximum time repairs could take (with all repairs done sequentially)

Decision Variables

I_r: interval variable representing the start and end times of repair job r

S_r: start time of repair job r. Determined by I_r

E_r: end time of repair job r. Determined by I_r

Formulation

$$\min(\max_{r \in R} E_r)$$

$$\text{noOverlap}_{r \in c}(\{I_r\}) \quad \forall c \in C \qquad \text{(C1)}$$

$$\max_{r \in R} E_r \leq T \quad \forall r \in R \qquad \text{(C2)}$$

$$S_r \geq 0 \quad \forall r \in R \qquad \text{(C3)}$$

Discussion

The objective function is the maximum end time for any road job, which we aim to minimize. C1 specifies that for each critical road subset, there can be no overlapping repair jobs for the roads in that subset. Note that initially, only critical road subsets of length 2 are used. However, with each iteration of the Master Problem, more subsets of higher lengths are added. C2 specifies that the maximum end time must be within the time horizon of the problem. And C3 specifies that all jobs must start at or after timestep 0.

3 Subproblem

Index Use

$n \in N$: index on nodes

- Node s: source node
- Node d: destination node

$(i, j) \in R$: index on road from node i to node j

$R' \subseteq R$: subset of roads currently being repaired

Data

$CAPACITY_{ij}$: capacity on arc (i, j)

$THRESHOLD$: lower limit for network capacity

Decision Variables

X_{ij}: flow assigned to arc between node i and j

Formulation

$$
\begin{aligned}
& \text{Assign } X_{ij} \\
\text{s.t} \quad & \sum_{(i,n)\in R} X_{in} = \sum_{(n,j)\in R} X_{nj} \quad \forall n \in N \backslash \{s, d\} && \text{(C1)} \\
& \sum_{(i,d)\in R} X_{id} \geq THRESHOLD && \text{(C2)} \\
& X_{ij} \leq CAPACITY_{ij} \quad \forall (i, j) \in R && \text{(C3)} \\
& X_{ij} = 0 \quad \forall (i, j) \in R' && \text{(C4)} \\
& X_{ij} \geq 0 \quad \forall (i, j) \in R && \text{(C5)} \\
& X_{ij} \in \mathbb{Z} \quad \forall (i, j) \in R && \text{(C6)}
\end{aligned}
$$

Discussion

This is a constraint satisfaction problem (CSP), so there is no objective function. After the Master Problem returns a repair schedule, we will determine the set of all unique combinations of roads that are being repaired at any time. For each of these combinations, we will run this Subproblem. The first constraint (C1) ensures that the net flow at each transshipment node is equal to zero. Constraint C2 ensures that the total capacity of the network is above the defined threshhold. Constraint C3 specifies that the flow along each arc must be less than or equal to the capacity of that arc. Constraint C4 specifies that all roads which are currently being repaired must have a flow of zero. Finally, constraints C5 and C6 ensure that all flows are nonnegative integers.

4 Pseudocode

4.1 Data Generation

```
Input:
    - NUM_NODES: Number of nodes in the network
    - BOUNDS: Geographical bounds for node placement
    - INITIAL_RADIUS, MAX_DISTANCE: Parameters for Poisson Disk Sampling
    - BIAS: Boolean flag to determine random capacity bias

Output:
    - GRAPH: Adjacency matrix with capacities
    - REPAIR_TIMES: Dictionary of repair times for roads
    - SOURCE_NODE, DESTINATION_NODE: IDs of the source and destination nodes

Steps:

Generate a random geographical layout of nodes using Poisson Disk Sampling
    - NUM_NODES held constant
    - Number of roads brought close to TARGET_ROADS by adjusting the radius

Compute GRAPH:
    - Connect nodes within MAX_DISTANCE.
    - Assign random capacities (biased towards larger or smaller numbers if specified)

Assign random REPAIR_TIMES to all roads

Determine SOURCE_NODE as bottom-leftmost node and DESTINATION_NODE as top-rightmost node

Save the data to CSV and JSON files
```

4.2 Preprocessing Step

```
Input:
    - GRAPH: Adjacency matrix
    - SOURCE_NODE, DESTINATION_NODE: Source and destination nodes
    - THRESHOLD: Minimum required max flow
Output:
    - CRITICAL_SUBSETS: List of critical length-2 road subsets whose removal reduces the max flow below

Steps:

CRITICAL_SUBSETS = []
For each road in GRAPH:
    - Temporarily remove the road from ADJ_MATRIX.
    - Check if max flow from SOURCE_NODE to DESTINATION_NODE falls below THRESHOLD by calling the SUBPRO
    - If true, add the road to CRITICAL_SUBSETS

Return CRITICAL_SUBSETS
```

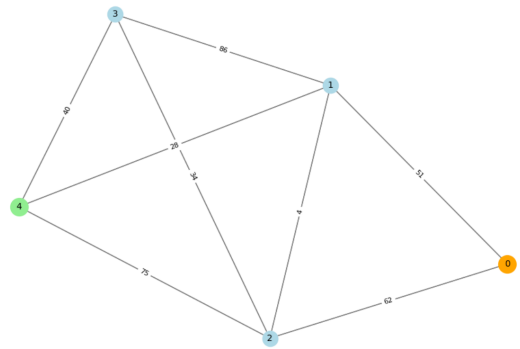
4.3 Master Problem

```
Input:
    - GRAPH: Adjacency matrix for graph
    - REPAIR_TIMES: Dictionary of repair times for roads
    - CRITICAL_SUBSETS: Precomputed critical subsets
    - TIME_HORIZON: Maximum allowable time for all repairs

Output:
    - SCHEDULE: Start and end times for each repair job

Steps:

Define the scheduling problem in CPlex:
    - Define an interval variable for each road repair job
    - For each subset in CRITICAL_SUBSETS:
        - Add no-overlap constraint for all roads in the subset
    - Set objective as the maximum end time of all jobs (minimize)

Set feasible_solution to FALSE

While feasible_solution is FALSE:
    - Solve model using CPlex
    - Find all overlaps in the repair schedule
    - For each OVERLAP:
        - NEW_GRAPH = Graph.Copy()
        - Remove the overlapping roads from NEW_GRAPH
        - Call the SUBPROBLEM on NEW_GRAPH to check feasibility
        - If infeasible:
            - Add OVERLAP to CRITICAL_SUBSETS
            - OPTIONAL: add large subsets of OVERLAP to CRITICAL_SUBSETS as a heuristic
            - Reset the Critical Subset Constraint to include all new overlaps
        - If feasible
            - Set feasible_solution to TRUE

Return the final SCHEDULE
```

4.4 Subproblem

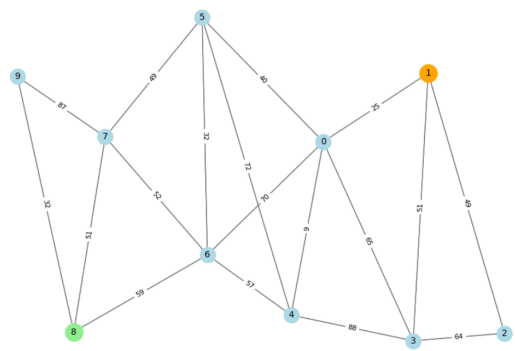
```
Input:
    - GRAPH: Adjacency matrix with capacities
    - SOURCE_NODE, DESTINATION_NODE: Source and destination nodes
    - THRESHOLD: Minimum required max flow

Output:
    - FEASIBLE: Boolean for whether the max flow from SOURCE_NODE to DESTINATION_NODE is at least equal

Steps:

# A truncated version of the Edmonds-Karp Algorithm

Create a directed graph (DIR_GRAPH) from GRAPH
DIR_GRAPH = Empty Graph
    - For each undirected edge (U, V) in GRAPH:
        - Add a directed edge (U,V) to DIR_GRAPH
        - Add a reverse edge (V, U) to DIR_GRAPH with the same capacity as (U, V)
Initialize:
    - FLOW_DICT: Dictionary to track of flow along each edge (initially 0 for all edges).
    - RESIDUAL: Graph.Copy()
    - MAX_FLOW: 0

Check whether a PATH exists from SOURCE to SINK in RESIDUAL using Breadth-First Search (BFS)

While a PATH exists:
    - Find the augmenting path (AUG_PATH) from SOURCE to SINK using Dijkstra's algorithm.
        - If no path exists:
            - Return FALSE.

    - Set PATH_FLOW to the minimum residual capacity of all edges along AUG_PATH.

    - For each edge (U,V) in PATH
        - Increase FLOW_DICT[U][V] by PATH_FLOW.
        - Decrease FLOW_DICT[V][U] by PATH_FLOW.
        - Reduce the residual capacity of edge (U, V) in RESIDUAL by PATH_FLOW.
        - Increase the residual capacity of reverse edge (V, U) in RESIDUAL by PATH_FLOW.
        - If the residual capacity of edge (U, V) becomes 0, remove the edge from RESIDUAL.

    Increase MAX_FLOW by PATH_FLOW.

    If MAX_FLOW >= THRESHOLD:
        - Return TRUE.

Return MAX_FLOW >= THRESHOLD
```

5 Test Data

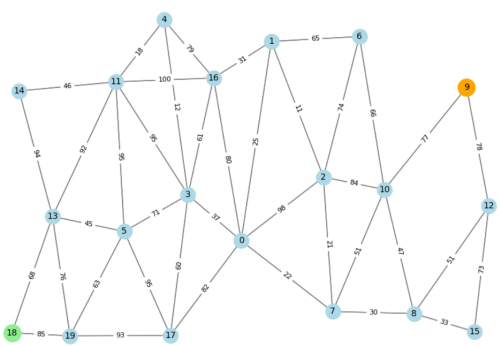


Nodes	Edges	Max Flow	Effective Max Flow
5	8	113	51

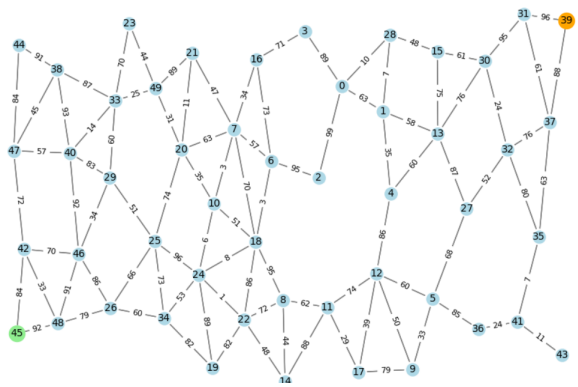
Note: Max flow is the maximum flow from the source to the destination nodes. *Effective Max Flow* is the maximum flow after removing one edge (since every road must be repaired, the capacity during the repair sequence will never be above this).



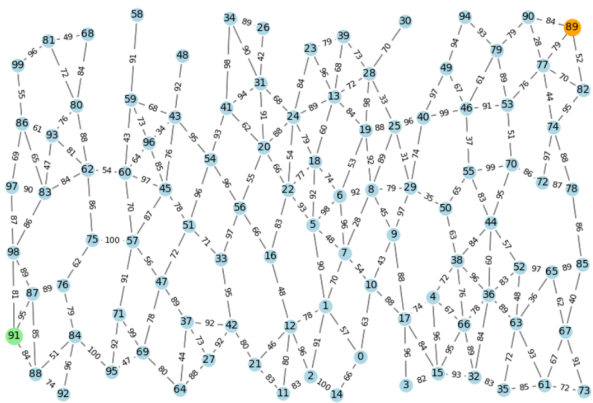
Nodes	Edges	Max Flow	Effective Max Flow
10	18	125	74



Nodes	Edges	Max Flow	Effective Max Flow
20	40	153	68



Nodes	Edges	Max Flow	Effective Max Flow
50	92	176	83



Nodes	Edges	Max Flow	Effective Max Flow
100	192	215	131

6 Results

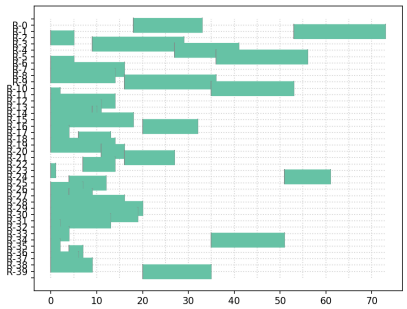
Here is a summary of the results:

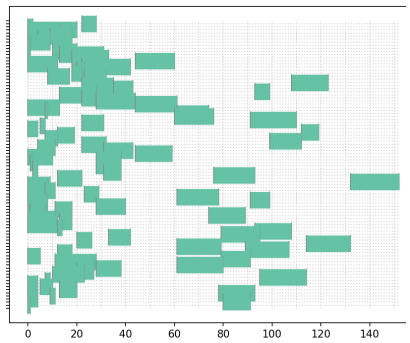
Number of Nodes	Effective Max Flow	Threshold Used	Sequential (Max) Time	Best Time Found	Best Bound	Optimality Gap	Number of Lazy Constraints Added	Time for Preprocessing (s)	Time for Solving (s)	Total Time (s)
5	51	51	68	24	24	0.00%	2	0.03	0.09	0.18
10	74	74	188	36	36	0.00%	561	0.12	114.43	114.55
20	68	68	429	73	66	9.59%	352	0.69	50.51	51.20
50	83	83	907	152	46	69.74%	1011	5.44	104.02	109.46
100	131	92	1907	73	20	72.60%	663	47.89	199	246.89

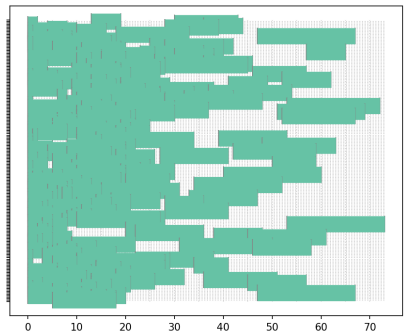
For each problem instance, the algorithm was run for between 0 and 5 minutes, and the best solution found was logged. I chose to stop after 5 minute, because I attempted to improve the optimality gap for each problem size, but the time would quickly grow from 5 minutes to 1 hour or more.

For problems of size ≤ 50, I was able to find optimal (or near-optimal) solutions while ensuring the capacity was always at or above the *Effective Max Flow*. However, this became infeasible and I had to relax the problem for the 100-node graph. Here I multiplied the Effective Max Flow by 0.7, ensuring that the network always stays at at least 70% of the maximum possible capacity (when removing one edge). As we can see, the optimality gap for this relaxed version was still higher than for the other instances.

Below are some sample schedules generated by the algorithm (for 20, 50, and 100 nodes, respectively).







7 Discussion

Scalability

One big barrier to scalability is the combinatorial explosion of the number of subsets of roads as graphs become larger. My algorithm currently just generates an optimal (or close to optimal) schedule, then checks for overlaps that violate the capacity constraint, and adds these overlaps as new constraints. The problem is that the number of possible overlaps grows exponentially (as 2^E, where E is the number of edges). For the 50-node graph, there are $4.95 \times 10^2 7$ possible road subsets. So without an effective heuristic to guide the scheduling process, it is almost impossible for this approach to scale to larger graphs. Moreover, the preprocessing step of enumerating the length-2 critical subsets requires calculating the max flow of $\binom{92}{2} = 4186$ graphs for the 50-node, 92-edge graph.. For 100 nodes and 192 edges, this number balloons to 18836 graphs. This step isn't essential, but without a "warm start" the optimizer struggles to converge to anything.

Complexity of Implementation

Breaking down the problem into a preprocessing step, a master problem, and a subproblem, where each part leverages existing methods (max flow algorithms, constraint programming, and max flow algorithms, respectively) makes the implementation not too difficult. However, this does require setting up lazy constraint generation, which can be a challenge in Cplex. Furthermore, I decided to implement the Edmonds-Karp algorithm from scratch as an educational challenge, but there are libraries like NetworkX than can automatically run it for you. If I had decided to implement BFS or Dijkstra's algorithm from scratch, it would have added to the complexity of implementation.

Sensitivity to Input Data

It was actually quite a challenge to generate input data for this problem, because there needs to be the right density of edges in the graph. Too many edges, and the problem becomes trivial (it's too easy to bypass roads in repair), and not enough edges makes the problem infeasible most of the time. Furthermore, a slight variation in the capacity of an edge is enough to convert a problem instance from feasible to infeasible. This doesn't mean that the process won't find an optimal or near-optimal solution. It just means that the results will be wildly different for different instances, although the solution quality will stay roughly even.

Convergence to Optimality

The algorithm is able to converge to an optimal solution within 2-3 minutes for small problem sizes (up to 20). However, it becomes difficult to find an optimal solution for larger problem sizes. In theory, there is nothing stopping the algorithm from converging to optimality. If I choose to set the "Optimality Gap Tolerance" parameter to 0, eventually all critical subsets will be enumerated and their constraints enforced, thus giving a certificate of optimality. However, this is no better than brute force enumeration.